

Experiment 4: Artificial Neural Network using Backpropagation

Aim:

To build an **Artificial Neural Network** using the **Backpropagation algorithm** and test it using a dataset.

Python Program:

```
import numpy as np
```

```
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
```

```
y = np.array([
    [0],
    [1],
    [1],
    [0]
])
```

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

```
def sigmoid_derivative(x):  
    return x * (1 - x)  
  
np.random.seed(1)  
  
w_hidden = np.random.uniform(size=(2, 2))  
b_hidden = np.random.uniform(size=(1, 2))  
w_output = np.random.uniform(size=(2, 1))  
b_output = np.random.uniform(size=(1, 1))  
  
learning_rate = 0.5  
epochs = 10000  
  
for i in range(epochs):  
    hidden_input = np.dot(X, w_hidden) + b_hidden  
    hidden_output = sigmoid(hidden_input)  
  
    final_input = np.dot(hidden_output, w_output) + b_output  
    predicted_output = sigmoid(final_input)  
  
    error = y - predicted_output  
  
    d_output = error * sigmoid_derivative(predicted_output)  
    error_hidden = np.dot(d_output, w_output.T)  
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)  
  
    w_output += np.dot(hidden_output.T, d_output) * learning_rate
```

```
b_output += np.sum(d_output, axis=0, keepdims=True) * learning_rate
```

```
w_hidden += np.dot(X.T, d_hidden) * learning_rate
```

```
b_hidden += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate
```

```
print("Predicted Output:")
```

```
print(np.round(predicted_output, 4))
```

```
print("\nClassified Output:")
```

```
print((predicted_output > 0.5).astype(int))
```

Output:

```
print((predicted_output
```

```
... Predicted Output:
```

```
[[0.0193]
 [0.9833]
 [0.9834]
 [0.0173]]
```

```
Classified Output:
```

```
[[0]
 [1]
 [1]
 [0]]
```
